Separate Before You Compress: The WWHO Tokenization Architecture

Kusal Darshana
Remeinium Research
thekusaldarshana@remeinium.com

March 4, 2026

Abstract

Current Large Language Models (LLMs) mostly use BPE (Byte Pair Encoding) based tokenizers, which are very effective for simple structured Latin scripts such as English. However, standard BPE tokenizers struggle to process complex Abugida scripts due to their structural complexity. The problem is that these tokenizers break complex conjuncts, which are multi-codepoint grapheme clusters, into meaningless sub-character units. This degrades the LLM's reasoning efficiency by forcing it to learn basic orthographic structures at inference time and raises inference costs, resulting in a significant "Token Tax" for the Global South. We propose a new three-layer architecture, the WWHO (Where-What-How Often), and an algorithm named SGPE (Syllable-aware Grapheme Pair Encoding) that separates the linguistic rules of the script from the statistical compression process while enabling seamless multilingual tokenization. Using Sinhala and Devanagari (Hindi/Sanskrit) as highly complex Abugida scripts, we trained WWHO on a cleaned 30-million-sentence dataset and evaluated on a 1,499,950-sentence test set. For Sinhala, SGPE achieves a Token to Word Ratio (TWR) of 1.274 with 4.83 characters per token, representing a 61.7% reduction in tokens compared to OpenAI's `o200k_base`. For Hindi, it achieves a TWR of 1.181 (27.0% reduction vs o200k). On the mixed-script (Sinhala, Devanagari, and English) dataset, SGPE achieves an overall TWR of 1.240, representing token reductions of 36.7%, 39.6%, and 60.2% relative to o200k_base, Llama 4 Scout, and DeepSeek V3, respectively. This effectively extends the usable context window by up to 4.38× for these Abugida languages while ensuring a Linguistic Zero-Breakage Guarantee, which ensures that no valid syllable is ever split across multiple tokens.

Keywords—Tokenizer, Token Tax, Abugida, Sinhala, Hindi, BPE, GPE, SGPE, WWHO, zero-breakage guarantee, NLP

1 Introduction

Tokenization plays a crucial role in defining a language model's performance[7] and reasoning capabilities. This helps the model understand the language; if it fails to capture the true morphological structure of the language, its reasoning and the quality of its responses in that language are also poor. When a tokenizer breaks a language's atomic grapheme clusters, it leads to three major issues beyond the side effects. First, it breaks the linguistic meaning of the cluster, blocks the model's linguistic understanding, then increases the fertility (the average number of tokens a tokenizer gener-

ates per word) and reduces the usable context window. Goldman et al. (2024) show that tokenizers' compression efficiency correlates with downstream performance[14]. Increasing the number of tokens decreases the LLM's reasoning capacity by distributing attention across more tokens. Third, since the Transformer's self-attention mechanism scales quadratically $O(N^2)$, a larger number of tokens increases inference cost. This affects the economy of both users and companies by imposing a "Token Tax"[1, 21]. Over a billion people globally use Abugida[1], pay more, but get less.

Byte Pair Encoding (BPE)[13, 23] is effective for English because its atomic unit is the byte. However, complex Abugida scripts are structured into grapheme clusters composed of elements such as conjuncts, diacritics (pili), virama (`U+0DCA`, ්), and Zero-Width Joiner (`U+200D`), etc., all of which utilize Unicode. Unicode characters frequently require two or more bytes, leading to BPE fragmentation in these scripts. For example, OpenAI's o200k_base tokenizer splits the Sinhala word "ආයුබෝවන්" into ['ආ', 'යු', 'බ', 'ෝ', 'ව', 'න්']. Such splits disrupt both the orthographic integrity and semantic coherence of the language. As a result, Sinhala and other Abugida scripts often exceed a 3.0 token-to-word ratio (TWR), whereas English typically achieves a TWR near 1.3. This discrepancy results in a reduction of up to 4.38× in the context window for these languages (section 5). Additionally, this fragmentation can lead to hallucinations due to LLMs' limited linguistic understanding. It's often observed that when LLMs generate text, they sometimes mix it with other scripts, and we hypothesize that this is caused by poor tokenization alongside poor data quality. Here are some examples: "पासල", "පරිගණකය".

In this work, using Sinhala and Devanagari (Hindi/Sanskrit) as highly complex Abugida scripts to demonstrate the architecture, we introduce **WWHO (Where-What-How Often)**, a three-layer architecture that prioritizes the linguistic integrity before applying any statistical compression. Layer 1, the Router, identifies script boundaries to route segments between Latin and script-specific processing units with $O(N)$ time complexity. Layer 2, named LinguisTrie, utilizes a table-driven Deterministic Finite Automaton (DFA) to group raw Unicode characters into complete, atomic syllables (defined in section 3). This layer guarantees that complex conjuncts or clusters are never broken into meaningless parts. We present a formal definition using regular expressions that represents the entire linguistic algorithm for how any syllable form is built from elements such as consonants, vowels, viramas (HAL), conjuncts, etc. (section 3). As layer 3, we extend the Grapheme Pair Encoding (GPE) concept by Velayuthan et al. (2025) [25] into Syllable-aware GPE (SGPE), which applies the statistical merging to the pre-defined syllables. WWHO further combines SGPE with standard BPE through a Unified Meta-Vocabulary to support code-switching without ID collisions. In this way, WWHO helps LLMs better understand the language by providing well-formed syllables and pass-through characters with a formal zero-breakage guarantee, freeing the language model from the burden of learning basic character reconstruction, restoring both linguistic integrity and computational efficiency at the source.

2 Background

The world is working to address this BPE fragmentation in Abugida scripts, as it has become a significant issue affecting over a billion people. Among recent works, Velayuthan et al. (2025) proposed Grapheme Pair Encoding (GPE)[25], which operates on Unicode grapheme clusters to avoid byte-level fragmentation. We extend the GPE into SGPE (Syllable-aware GPE) by adding a syllabification layer called LinguisTrie, which utilizes a DFA (Deterministic Finite Automaton) to

[1] `https://www.unicode.org/events/utw/2023/talks/abugida/`

ensure perfect syllable extraction before statistical compression. And Senanayake et al published an enhanced, punctuation-based Sinhala tokenizer at the 2019 National Information Technology Conference (NITC)[22]. Recent efforts to improve Sinhala LLMs, such as SinLlama (Aravinda et al., 2025), focused on enhancing the Llama-3-8B tokenizer with Sinhala-specific vocabulary[8].

Historically, in the Sinhala NLP, Weerasinghe et al. (2005) have established a rule-based syllabification algorithm for Sinhala[26]. A broader record of such developments is documented in the survey by de Silva (2019) [11], which includes the Sinling library[3], a toolkit for various Sinhala NLP tasks, including tokenization. However, these traditional methods are often hardcoded for a specific script. WWHO focuses on a script-independent approach, allowing any Abugida or complex script type to be dynamically injected, without changing the code.

In addition to these works, Clark et al. (2022) presented a tokenization-free encoding concept, focusing on efficiency and tokenization-free modeling[9]. A study by Dagan et al. (2024) explores several pre-tokenization approaches based on the BPE architecture[10]. However, it was mainly focused on English, code, and general multilingual tokenization. Among related works, Maung & Mikami (2008) have proposed a rule-based syllabification approach that utilizes FSA (Finite State Automaton) to demonstrate the syllable structure of Myanmar script[18]. Htay et al (2008) have presented a Word Segmentation using syllable-level longest matching for the Myanmar language[15]. Although these efforts often mainly focus on ensuring the linguistic accuracy within a single language, they lack the real-world multilingual and code-switching (cross-script) tokenization required by frontier LLMs. WWHO fulfills this gap by providing a unified framework that can be integrated with existing systems, without deprecating or replacing them.

3 A Regular-Language Framework for Abugida Syllables

Most Abugida scripts share a common linguistic structural invariant: consonants possess an inherent vowel, which is either suppressed by a virama (HAL) or modified by diacritics. Complex conjuncts are formed through the interaction of viramas and Zero-Width Joiners (ZWJ). We hypothesize that syllable segmentation for this family can be formalized as a regular language.

Definition 1. A perfect, atomic orthographic syllable in a linguistic script is further indivisible, and if divided, one or more segments will become individually meaningless.

3.1 Character Classes

We define the following distinct abstract classes based on orthographic roles for Sinhala and Devanagari(Hindi/Sanskrit).

Table 1: Sinhala Unicode Character Class

Class	Unicode Range	Description
C (consonant)	U+0D9A–U+0DC6	Base consonants (vyanjana)
V (vowel)	U+0D85–U+0D96	Independent vowels (svara)
P (pili)	U+0DCF–U+0DDF, U+0DF2–U+0DF3	Dependent vowel signs (diacritics)
H (HAL)	U+0DCA	Virama (vowel suppressor / "Hal-Lakuna")
Z (ZWJ)	U+200D	Zero-Width Joiner (conjunct connector)
M (post-modifier)	U+0D82, U+0D83	Anusvara and visarga (nasalization and aspiration markers)
O (other)	All else	Non-Sinhala passthrough

3.1.1 Formal Definition for Sinhala:

A valid Sinhala syllable can be defined over the alphabet $\Sigma_s = \mathbf{C} \cup \mathbf{V} \cup \mathbf{P} \cup \mathbf{H} \cup \mathbf{Z} \cup \mathbf{M}$ matching the regular expression

$$S_s = C(HZ?C)^*(P \mid H)?M? \quad | \quad VM?\,.$$

This definition captures all linguistically valid syllable forms that consist of independent vowels, conjuncts, viramas (HAL), ZWJs, diacritics, and optional post-modifiers.

Table 2: Devanagari Unicode Character Class

Class	Unicode Range	Description
C (consonant)	U+0915–U+0939, U+0958–U+095F, U+0978–U+097F	Base consonants (including nukta-precomposed)
V (vowel)	U+0904–U+0914, U+0960–U+0961, U+0972–U+0977	Independent vowels
P (matra)	U+093E–U+094C, U+094E–U+094F, U+0955–U+0957, U+0962–U+0963	Dependent vowel signs (matras)
H (halant)	U+094D	Virama (vowel suppressor / conjunct connector)
Z (ZWJ/ZWNJ)	U+200D, U+200C	Zero-Width Joiner and Non-Joiner
N (nukta)	U+093C	Nukta (diacritic dot for borrowed phonemes)
M (modifier)	U+0900–U+0903, U+093D, U+0950–U+0954, U+1CD0+, U+A8E0+	Anusvara, chandrabindu, visarga, avagraha, Vedic accents
O (other)	All else	Non-Devanagari passthrough

Formal Definition for Devanagari: A valid Devanagari syllable is defined over the alphabet $\Sigma_d = \mathbf{C} \cup \mathbf{V} \cup \mathbf{P} \cup \mathbf{H} \cup \mathbf{Z} \cup \mathbf{N} \cup \mathbf{M}$ matching the regular expression:

$$S_d = CN?(HZ?CN?)^*(P \mid H)?M^* \mid VM^* \tag{1}$$

The Devanagari grammar also incorporates the Nukta (N) diacritic. Unlike Sinhala, Devanagari, especially Vedic texts, use multiple modifiers; this definition allows for having M^*.

Pass-through tokens: Any character $c \notin \Sigma$ is considered as under the set O (Other) for both alphabets and is treated as a pass-through token to handle noisy or invalid Unicode text.

3.2 The Valid Language Schema

We define the requirements for a valid language schema to ensure the structural integrity of the syllabification process.

Definition 2. A language schema is considered **valid** if it satisfies the following conditions.

1. **Disjoint Character Classes:** All defined character classes within the schema must be disjoint. No Unicode codepoint should be assigned to more than one named class simultaneously.

2. **Completeness:** The schema must declare at least one non-empty Unicode block range. Every codepoint assigned to a named class must fall within that declared block. Codepoints within the block that are not assigned to any named class are implicitly assigned to class O, ensuring no codepoint in the script block is left unclassified.

3. **Determinism:** The DFA transition function is deterministic, which means that each state has at most one transition for a given class. Undefined transitions (marked as '—') are explicit boundary signals used to trigger the emission of the last accepted syllable.

4. **Emit State Isolation:** Both emit states, ORPHAN and PASSTHROUGH, must have no outgoing transitions, and must only be reachable directly from the START state. This ensures that the emit states produce only one single string, before the scanner continues to the START state to extract the next syllable.

5. **Grammar Alignment:** The language recognized by the DFA via its set of accept states must be identical to the language generated by the formal grammar S defined within the schema (e.g., S_s or S_d).

6. **Maximal-Munch Consistency:** The DFA must operate as a Maximal Munch (greedy longest-match) scanner. Instead of stopping at the first valid syllable boundary, the machine must continue to evaluate subsequent characters and extract the longest possible valid syllable.

   e.g: When considering the string 'ක්‍රෝ', it is made up as: ක + ් + [ZWJ] + ර + ෝ. Instead of stopping at the syllable `ක' and extracting it, the DFA must continue to grab the full syllable `ක්‍රෝ'.

3.3 The Language is Regular

Proposition 1. *Most Abugida scripts, including Sinhala and Devanagari, can be represented by regular expressions, and therefore, they are regular languages.*

Proof. Because S (S_s or S_d) is a regular expression and regular expressions define regular languages.

We present the following DFA (Deterministic Finite Automaton) transition tables, declared according to the 'Valid Language Schema' conditions as defined, for Sinhala and Devanagari.

Table 3: DFA Transition Table for Sinhala Syllable Recognition

State	C	V	H	P	Z	M	O
START	IN_CLUSTER	IN_VOWEL	ORPHAN	ORPHAN	ORPHAN	ORPHAN	PASSTHROUGH
IN_CLUSTER	—	—	HAL_SEEN	PILI_SEEN	—	ACCEPT	—
HAL_SEEN	IN_CLUSTER	—	—	—	ZWJ_SEEN	ACCEPT	—
ZWJ_SEEN	IN_CLUSTER	—	—	—	—	—	—
PILI_SEEN	—	—	—	ORPHAN	—	ACCEPT	—
IN_VOWEL	—	—	—	—	—	ACCEPT	—
ACCEPT	—	—	—	—	—	—	—

Legend: C = Consonant, V = Independent Vowel, H = Hal/Virama, P = Dependent Vowel (Pili), Z = Zero-Width Joiner, M = Modifier (Anusvara/Visarga), O = Other.
— = No valid transition (syllable boundary signal)

Table 4: DFA Transition Table for Devanagari Syllable Recognition

State	C	V	H	P	Z	N	M	O
START	IN_CLUSTER	IN_VOWEL	ORPHAN	ORPHAN	ORPHAN	ORPHAN	ORPHAN	PASSTHROUGH
IN_CLUSTER	—	—	HAL_SEEN	PILI_SEEN	—	NUKTA_SEEN	IN_CLUSTER_M	—
NUKTA_SEEN	—	—	HAL_SEEN	PILI_SEEN	—	—	IN_CLUSTER_M	—
HAL_SEEN	IN_CLUSTER	—	—	—	ZWJ_SEEN	—	ACCEPT	—
ZWJ_SEEN	IN_CLUSTER	—	—	—	—	—	—	—
PILI_SEEN	—	—	—	ORPHAN	—	—	IN_CLUSTER_M	—
IN_CLUSTER_M	—	—	—	—	—	—	IN_CLUSTER_M	—
IN_VOWEL	—	—	—	—	—	—	IN_VOWEL_M	—
IN_VOWEL_M	—	—	—	—	—	—	IN_VOWEL_M	—
ACCEPT	—	—	—	—	—	—	—	—

Legend: C = Consonant, V = Independent Vowel, H = Virama/Halant, P = Dependent Vowel (Matra), Z = Zero-Width Joiner/Non-Joiner, N = Nukta, M = Modifier (Anusvara/Chandrabindu/Visarga etc.), O = Other.
— = No valid transition (syllable boundary signal)

The DFA identifies a syllable boundary using the states. The accepting states for Sinhala are IN_CLUSTER, IN_VOWEL, HAL_SEEN, ZWJ_SEEN, PILI_SEEN, and ACCEPT. For Devanagari, the set is extended to include NUKTA_SEEN, IN_CLUSTER_M, and IN_VOWEL_M.

3.4 Zero-Breakage Guarantee

Definition 3. A tokenizer T satisfies the Zero-Breakage Guarantee for a given language schema with grammar S if it satisfies two conditions. Let $T(w) = t_1, t_2, t_3, ..., t_n$ be the sequence of tokens produced for an input string w.

1. Losslessness: The concatenation of all tokens must perfectly reconstruct the original string w ($w = t_1 \cdot t_2 \cdot t_3 ... \cdot t_n$), with the exception that any character missing from the vocabulary or removed during pruning is replaced with the `[UNK]` token.

2. Linguistic Integrity: Each token t_i is composed of one or more complete atomic syllables accepted by S, or a single passthrough character (class O or $ORPHAN$).

4 The WWHO Architecture

The WWHO breaks down the tokenization process into first principles using three layers, based on the concepts of Where, What, and How Often. This approach enables more effective training of multilingual tokenizers by addressing Abugida issues and seamlessly integrating with current frontier tokenization approaches, without replacing existing methods. The architecture is completely language independent. The linguistic constraints and formal grammar of a specific script, such as the character class, the DFA transition table, and Unicode block ranges, are separated from the core algorithm and defined dynamically via an external `JSON` Schema, allowing WWHO to support any Abugida or complex script without modifying the codebase, simply by swapping the schema file. To add a new linguistic script, it's enough to add only a new language-specific `JSON` file. Both Router and LinguisTrie use the same schema files.

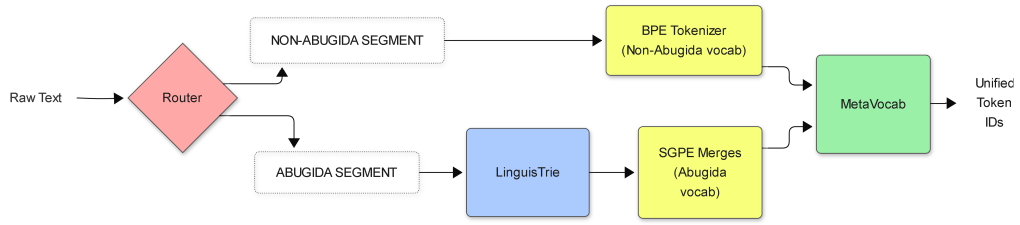

Figure 1: WWHO architecture.

4.1 Layer 1: Router (The "Where")

The real-world data are multilingual code-switching text, rather than pure monoscripted text. Since BPE is highly effective for Latin scripts such as English, we don't deprecate it. Instead, the WWHO implements a targeted, localized routing layer that scans the text in a single pass using a Unicode range block scanner. The scanned text is cleanly partitioned into Abugida and Non-Abugida segments. In this partitioning, the router detects leading spaces and flags them for the next LinguisTrie layer to attach to. Once segmented, the text is routed to its respective specialists.

4.1.1 Unicode-based Script Identification

To demonstrate the architecture, we use 3 different script types as follows.

1. Sinhala: `[U+0D80]-[U+0DFF]`
2. Devanagari: `[U+0900]-[U+097F]`
3. Other: which do not belong to the above two categories, such as Latin/ASCII/Code/Digits/Emojis, etc.

Symbols such as Devanagari Punctuation (Dandas - `(U+0964)`, `(U+0965)`) are also considered as native characters and are included in the relevant block.

4.1.2 Script Boundaries

The router performs an O(N) linear scan over the text, classifying each character in O(1) time to identify script transitions. It enforces a Hard Script Boundary even in the absence of whitespace, resulting in an explicit partitioning of text into distinct language segments.

e.g., 'ᎤᏪpple' is segmented into 'ᎤᏪ' and 'pple'

ZWJ/ZWNJ Handling: Since complex conjuncts in such Abugida scripts are made up with invisible Unicode characters such as Zero-Width-Joiner (U+200D) and Zero-Width-Non-Joiner (U+200C) can cause incorrect script segmentation. But our router explicitly absorbs such joiners into the correct segment, preventing incorrect script boundaries. This further assists the next layer, LinguisTrie, in performing accurate syllabification, and it's fundamental to ensuring Zero Breakage Guarantee.

4.1.3 Routing Policy

Each segment is then routed to the appropriate tokenizer. Then the SGPE encodes Abugida segments, while other Non-Abugida segments are encoded by the standard BPE. This decoupling enables SGPE integration with existing Latin-specialized BPE tokenizers while preserving the linguistic integrity of complex Abugida scripts.

4.2 Layer 2: LinguisTrie (The "What")

The LinguisTrie, a DFA-based lexical scanner, is the layer responsible for grouping the Abugida text segments sent by the router, according to their Unicode codepoints, into Atomic Orthographic Syllables following the DFA and grammar defined in Section 3. It operates with $O(N)$ time complexity by performing a single linear scan over the text.

4.2.1 The Syllabification Algorithm

Leading space handling: LinguisTrie captures the leading space flagged by the router and attaches it to the first identified syllable in a string. It attaches only one space character, and other whitespace (tabs, newlines) are emitted standalone.

Proposition 2. *The LinguisTrie correctly extracts Atomic Orthographic Syllables from any valid Unicode string defined via a valid language schema as defined in Section 3.2 (Definition 2).*

Proof. LinguisTrie strictly follows the mechanism below to extract perfect, atomic orthographic syllables from a given string:

1. **Direct Grammar Mapping:** Every transition of the DFA is designed to follow the syllable grammar, the Regex (S_s or S_d) defined within the script. Hence, the machine only moves forward if a syllable can be extracted from the given string.

2. **Maximal Munch (Greedy Longest Match):** The machine doesn't stop immediately once it finds a syllable. Instead, it continues to check whether the syllable can have a further expanded form.

3. **last_accept_pos:** The machine remembers the last accept state at which it found a valid transition according to the grammar. When moving forward, it never fails if it meets an invalid character. It moves back and emits the last valid syllable. Hence, the syllable is always a valid atomic orthographic syllable.

4. **Passthrough and Orphan handling:** If it finds a character belonging to class O (Other), which is outside of the script's alphabet or if it finds an $ORPHAN$, which can be a part of a syllable, but individually meaningless (eg: an individual HAL sign), it emits that as a single string to ensure the Losslessness principle.

□

Example Sinhala Syllabification

Input : චන්ද්‍රයාගේ ආලෝකය පෘථිවියට ක්ෂණිකව නොලැබේ.

Syllables : [`ච', `න්ද්‍ර', `යා', `ගේ', `ආ', `ලෝ', `ක', `ය',
`පෘ', `ථි', `වි', `ය', `ට', `ක්ෂ', `ණි', `ක', `ව', `නො', `ලැ', `බේ', `.']

Example Devanagari Syllabification

Input : विद्यालय में पढ़ाई होती है।

Syllables : [`वि', `द्या', `ल', `य', `में', `प',
`ढ़', `ई', `हो', `ती', `है', `।']

4.3 Layer 3: SGPE and Meta Vocabulary

4.3.1 Syllable-aware Grapheme Pair Encoding

While SGPE only merges the well-extracted, word boundary-aware atomic Abugida syllable pairs, which are also Unicode grapheme clusters, all non-Abugida text segments are processed by an existing BPE algorithm, and, for the demonstration, we used OpenAI's o200k_base as the BPE tokenizer. Velayuthan et al. (2025) introduced the GPE (Grapheme Pair Encoding) concept by replacing BPE's atomic unit (the byte) with Unicode grapheme clusters. SGPE extends this principle one level further: the atomic unit is a perfect Atomic Orthographic Syllable as defined in Section 3. Other than the following differences, SGPE is the same BPE algorithm:

1. **Syllabic initialization:** The base Abugida vocabulary is initialized with atomic syllables and passthrough characters emitted by LinguisTrie.

2. **Boundary-aware scoping:** Unlike BPE, SGPE merges consider the word boundaries by any non-Abugida characters, punctuations, digits, in addition to white spaces.

3. **Pruning:** To ensure the vocabulary allows for more unique and frequently used merges, SGPE implements a pruning mechanism that replaces syllables below a defined threshold, the prune frequency, as '`[UNK]`' tokens. The result is that any Unicode sequence that appears less than the prune frequency is not included in the vocabulary, keeping it cleaned from most malformed Unicode sequences or typographical noise and valid but extremely rare tokens.

Proposition 3. *The WWHO architecture ensures the Zero-Breakage Guarantee.*

Proof. Since the Router performs only a simple partitioning operation on a given string w without any deletion or modification, the character sequence is preserved. Then, as a complete scanner, LinguisTrie ensures no character is discarded at the syllabification stage. Since the SGPE layer only performs the merging on these units without modifying the characters, the Losslessness requirement ($w = t_1 \cdot t_2 \cdot t_3 ... \cdot t_n$) is satisfied (subject to `[UNK]`).

Inheriting Proof of Proposition 2 made in Section 4.2, SGPE only merges either perfect Atomic Orthographic Syllables accepted by S or fallback strings (Orphan/Passthrough). This satisfies the 2nd requirement for the Zero-Breakage guarantee, and the proposition follows. □

4.4 Unified Meta-Vocabulary

We introduce the Unified Meta-Vocabulary approach to prevent Abugida tokens processed by SGPE and non-Abugida tokens processed by BPE IDs from colliding with each other.

Let V_{BPE} be the vocabulary size of the BPE tokenizer (e.g., $V_{BPE} = 200,019$ for `o200k_base`), and V_{SGPE} be the size of the tokenized SGPE vocabulary. The WWHO architecture sequentially maps the ID spaces into a contiguous array:

$$\text{ID } 0 \ldots (V_{BPE} - 1) \longrightarrow \text{non-Abugida (via BPE)}$$
$$\text{ID } V_{BPE} \ldots (V_{BPE} + V_{SGPE} - 1) \longrightarrow \text{Abugida (via SGPE)}$$

During tokenization, a V_{BPE} offset is applied to all SGPE-generated IDs to align them within the unified meta-vocabulary. This prevents ID collisions and enables the model to deal with multiple scripts in a single embedding space. In inference, the process is reversed for detokenization.

4.4.1 End-to-End Execution Trace

To illustrate the complete workflow, consider the code-switched input: "ඔයා 1 special अद्भुत".

1. **Router (Layer 1):** The input is partitioned into script-specific segments: the Sinhala segment "ඔයා", the Other segment " 1 special", and the Devanagari segment " अद्भुत". The leading space before the Devanagari word is flagged for internal attachment.

2. **LinguisTrie (Layer 2):** The Abugida segments are decomposed into atomic orthographic syllables. The Sinhala segment produces `["ඔ", "යා"]` and the Devanagari segment produces `[" अ", "द्भु", "त"]`.

3. **SGPE and BPE (Layer 3):** Final tokens are generated based on the learned vocabulary. Non-Abugida segments are processed via the foundation BPE, while Abugida syllables are merged by SGPE.

The final tokenization result is:

```
["ඔයා", " ", "1", " special", " अद्भुत"]
```

5 Training & Evaluation

In this section, we evaluate the performance of the WWHO architecture across three key scripts (Sinhala, Devanagari, and English) and compare it against frontier tokenizers: OpenAI's `o200k_base`, Meta's `Llama 4 Scout`, and `DeepSeek V3`. We report the Token-to-Word Ratio (TWR), the characters per token (CPT), and the relative token reduction.

For training and evaluation, we created a 30-million sentence corpus[6] aggregating four open-source datasets to represent a balanced mix of Sinhala (35%), Hindi(45%), and English(20%). The final corpus consists of:

1. MADLAD_CulturaX_cleaned[12]

2. A filtered subset of CC100-Sinhala[24] containing only Sinhala and English code-mixed sentences.

3. Two subsets of MADLAD-400 (pure Hindi and pure English)[17]

4. A filtered subset of HINMIX_hi-en[16] containing only Hindi and English code-mixed sentences.

The corpus was split 95%/5% into training and test sets, yielding a 1,499,950-sentence evaluation partition containing 122 million characters.

5.1 Training Configurations

We set the training hyperparameters as follows, and the evaluation results are based on these configurations.

Table 5: SGPE Training Hyperparameters

Parameter	Value
Unified Meta-Vocabulary Size	328,020
BPE (o200k_base) Vocab Size	200,019
SGPE Vocab Size	128,000†
Prune frequency threshold θ	100
Training corpus size	∼28.5 M sentences
Evaluation corpus size	∼1.5 M sentences
Code-switching routing	Enabled

†**Note:** The SGPE vocabulary consists of 128,000 tokens, as the hyperparameter is set to 128,000. Since the LinguisTrie attaches space as a prefix to the next syllable, the standalone space token is not included in the SGPE vocabulary. Therefore, one space token is injected later, as an architectural design to ensure the system's integrity. Hence, the total SGPE vocabulary size is 128,001 tokens.

5.2 Tokenization Efficiency (TWR, CPT & Reduction)

The Token-to-Word Ratio (TWR) measures how many tokens are required to represent a single word on average. A TWR closer to 1.00 indicates that the tokenizer is effectively treating each word as a single unit without any fragmentation.

Table 6: Tokenization Efficiency Comparison Across Scripts

Script	Tokenizer	Total Tokens	TWR	CPT	% Reduction (vs SGPE)
Sinhala	SGPE (WWHO)	6,654,288	1.274	4.83	—
	OpenAI (o200k)	17,360,196	3.324	1.85	61.7%
	Llama 4 Scout	18,157,707	3.476	1.77	63.4%
	DeepSeek V3	29,152,698	5.581	1.10	77.2%
Hindi	SGPE (WWHO)	13,433,554	1.181	4.29	—
	OpenAI (o200k)	18,394,075	1.617	3.13	27.0%
	Llama 4 Scout	19,566,121	1.720	2.94	31.3%
	DeepSeek V3	31,682,218	2.786	1.82	57.6%
English	SGPE (WWHO)	7,240,147	1.330	4.46	—
	OpenAI (o200k)	7,420,527	1.364	4.35	2.4%
	Llama 4 Scout	7,512,843	1.381	4.30	3.6%
	DeepSeek V3	7,904,670	1.453	4.09	8.4%
Overall	SGPE (WWHO)	27,327,989	1.240	4.47	—
	OpenAI (o200k)	43,174,798	1.959	2.83	36.7%
	Llama 4 Scout	45,236,671	2.053	2.70	39.6%
	DeepSeek V3	68,739,586	3.119	1.78	60.2%

Table 7: Example Tokenizations Across Tokenizers

Word	Tokenizer	Tokens	Count
වියාකරණය	SGPE (Ours)	['වියා', 'කරණය']	2
	OpenAI (o200k)	['වි', 'යා', 'ක', 'රණ', 'ය']	5
	Llama 4 Scout	['වි', 'යා', 'කර', 'ණය']	4
	DeepSeek V3	['ව', 'ි', 'ය', 'ා', 'ක', 'ර', '^ණ', '^ය', 'ය']	9
ශ්‍රී ලංකාව	SGPE (Ours)	['ශ්\u200dරී', ' ලංකාව']	2
	OpenAI (o200k)	['ශ්', '\u200dරී', ' ලංක', 'ාව']	4
	Llama 4 Scout	['ශ්', '\u200dර', '[ී]', ' ල', 'ං', 'ක', 'ාව']	7
	DeepSeek V3	['^ශ', '^ර', '[ී]', '\u200d', 'ර', '[ී]', '^ල', '[ං]', '^ක', 'ා', 'ව']	13
अंतर्राष्ट्रीय	SGPE (Ours)	['अंतर्राष्ट्रीय']	1
	OpenAI (o200k)	['अ', 'ंतर', '्र', 'ाष्ट्रीय']	4
	Llama 4 Scout	['अ', 'ंतर', '्र', 'ाष्ट्रीय']	4
	DeepSeek V3	['अ', 'ंत', 'र', '्र', 'ाष', '्ट', '्री', 'य']	8
कृत्रिम बुद्धिमत्ता	SGPE (Ours)	['कृत्रिम', ' बुद्धिमत्ता']	2
	OpenAI (o200k)	['क', 'ृ', 'त्र', 'िम', ' बुद्ध', 'िम', 'त्ता']	7
	Llama 4 Scout	['क', 'ृ', 'त्र', 'िम', ' ब', 'ुद्ध', 'िम', 'त्ता']	8
	DeepSeek V3	['क', 'ृ', 'त्र', 'िम', ' ब', 'ुद', '्ध', 'िम', 'त्त', 'ा']	10

5.3 Key Observations

The "Token Tax" Reduction: For Sinhala text, SGPE demonstrates a massive 61.7%-77.2% reduction in token count, allowing 4.83 characters per token. While frontier models often fragment a single Sinhala word into 3-10 tokens with meaningless sub-word pieces, SGPE maintains a TWR of 1.274 while ensuring the Zero-Breakage Guarantee. And for Devanagari (Hindi), SGPE achieves 1.181 TWR and 4.29 CPT, reducing 27.0%-57.6% token reduction.

Overall Efficiency: Across a mixed multilingual test set containing Sinhala, Hindi, and English text in a 35%:45%:20% ratio, SGPE reduces the total token count by 36.7%-60.2% compared to the industry standard frontier tokenizers.

Reduction on English?: An unexpected observation during evaluation was the 2.4%-8.4% reduction in token count for the English compared to other tokenizers. While WWHO routes Latin text directly to the foundation BPE tokenizer, the observed efficiency gain is a result of unintentional code-switching within real-world datasets. **The Router's Contribution:** In datasets classified as English, there can be a minor percentage of Abugida texts(e.g., proper nouns, locations, or localized terms). While the foundation BPE tokenizer fragments these foreign segments into byte-level tokens, the WWHO's Router accurately isolates these Abugida clusters. Then LinguisTrie and SGPE perfectly tokenize them. This localized efficiency results in the observed token reduction even in English-dominant contexts.

To verify this observation and that the routing layer does not modify English segments while processing, we conducted a pure ASCII Stress Test using 10,000 sentences containing zero Abugida characters. In this controlled environment, the token counts produced by WWHO and o200k_base were 100% identical (0.0% difference), which means all the ASCII text was processed only by the BPE tokenizer.

5.4 Context Window Capacity Analysis

One of the most impactful factors of the efficiency of a tokenizer is the LLM context window. The context window capacity multiplier, which measures the context window saving by a tokenizer compared to another, can be calculated by the following formula.

$$\text{Capacity Multiplier} = \frac{Tokens_{baseline}}{Tokens_{sgpe}}$$

Table 8: Effective Context Window Capacity Multiplier Comparison

Script	Tokenizer	SGPE Tokens	Other Tokens	Context Multiplier
Sinhala	vs OpenAI (o200k)	6,654,288	17,360,196	2.61×
	vs Llama 4 Scout	6,654,288	18,157,707	2.73×
	vs DeepSeek V3	6,654,288	29,152,698	4.38×
Hindi	vs OpenAI (o200k)	13,433,554	18,394,075	1.37×
	vs Llama 4 Scout	13,433,554	19,566,121	1.46×
	vs DeepSeek V3	13,433,554	31,682,218	2.36×
English	vs OpenAI (o200k)	7,240,147	7,420,527	1.02×
	vs Llama 4 Scout	7,240,147	7,512,843	1.04×
	vs DeepSeek V3	7,240,147	7,904,670	1.09×
Overall	vs OpenAI (o200k)	27,327,989	43,174,798	1.58×
	vs Llama 4 Scout	27,327,989	45,236,671	1.66×
	vs DeepSeek V3	27,327,989	68,739,586	2.52×

As shown in our evaluation, SGPE provides a very effective context window capacity expansion for Abugida scripts. This allows over a billion Abugida users to use LLMs more effectively and get better responses. **The Token Tax** will have a significant drop via the WWHO architecture.

5.5 Vocabulary Quality & Glitch Token Prevention

Standard BPE tokenizers often allocate vocabulary capacity to statistical anomalies, leading to "glitch tokens" or orphaned Unicode joiners (e.g., isolated ZWJ or Viramas). In Abugida scripts, the generation of such artifacts by an LLM causes low response quality. To evaluate vocabulary quality, we conducted a Glitch Token detection test across the 1.5-million sentence evaluation corpus. The results confirm that SGPE produces 0 glitch tokens for both Sinhala and Devanagari scripts. Out of 128,001 SGPE tokens (Sinhala and Devanagari), only 1,394 (roughly 1%) remained unused during evaluation. This rate shows that by enforcing syllable-level linguistic accuracy, the SGPE algorithm avoids creating meaningless fragments and effectively uses its vocabulary for real-world multilingual script structures.

5.6 Empirical Validation of Zero-Breakage Guarantee

To verify the Zero-Breakage Guarantee defined and formally proven in Sections 3 and 4, we conducted a Round-Trip test using the same test corpus. Across 122 million mixed multilingual characters, SGPE achieved 100% round-trip consistency with zero non-[UNK] mismatches, practically verifying the theoretical proof.

6 Conclusion

In this work, we presented the WWHO, a novel DFA-based multilingual tokenization architecture, and SGPE, the merging algorithm used in WWHO.

We proposed that the basic, atomic unit of a linguistic script should be the Syllable, extending the GPE concept to SGPE. By separating linguistic rules from statistical compression, we ensure that complex grapheme clusters remain intact throughout the process.

WWHO satisfies the Zero-Breakage Guarantee both theoretically and practically, by ensuring that meaningful syllables are never fragmented. Our evaluation shows that SGPE achieves up to a 77.2% reduction in token count for Sinhala and 57.6% for Hindi compared to frontier tokenizers. This efficiency allows 4.29-4.83 characters per token and expands the effective context window by up to 4.38×. These results prove that preserving linguistic integrity directly reduces the "Token Tax" for Abugida languages.

We plan to scale the WWHO framework to support Abugida and other complex scripts. The full source code, evaluation harness, and developer notes are open-sourced on GitHub[4], and the full dataset we used to train and evaluate is available at Hugging Face[6]. Pre-trained artifacts, including the `vocab.json`, `tokenizer.json`, and optimized merge rules, are available via Hugging Face[5].

Acknowledgements

The author is deeply grateful to Dr. Nisansa de Silva for all the invaluable support and guidance on this research. We thank the Polyglots FYP[12], Allen AI[17], Mr. Kartik[16], and Mr. Muxin Tian[24] for open-sourcing the datasets we used to create ours[6]. We also acknowledge OpenAI[20], Meta[19], and DeepSeek[2] for their contributions by open-sourcing frontier tokenizers, which served as essential baselines for our evaluation.

References

[1] Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R Mortensen, Noah A Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9904–9923, 2023.

[2] DeepSeek AI. Deepseek-v3. `https://huggingface.co/deepseek-ai/DeepSeek-V3`, 2024. Hugging Face model card.

[3] Remeinium AI. Sinling library. `https://github.com/ysenarath/sinling`, 2019. Github Repository.

[4] Remeinium AI. Wwho source code. `https://github.com/remeinium/WWHO`, 2026. Github Repository.

[5] Remeinium AI. Wwho tokenizer. `https://huggingface.co/Remeinium/WWHO`, 2026. Hugging Face model card.

[6] Remeinium AI. Wwho-30m sinhala-hindi-english corpus. `https://huggingface.co/datasets/Remeinium/WWHO_30m`, 2026. Hugging Face model card.

[7] Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Buschhoff, et al. Tokenizer choice for llm training: Negligible or crucial? In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3907–3924, 2024.

[8] HWK Aravinda, Rashad Sirajudeen, Samith Karunathilake, Nisansa de Silva, Rishemjit Kaur, and Surangika Ranathunga. Sinllama-a large language model for sinhala. In *2025 Moratuwa Engineering Research Conference (MERCon)*, pages 617–622. IEEE, 2025.

[9] Jonathan H Clark, Dan Garrette, Iulia Turc, and John Wieting. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

[10] Gautier Dagan, Gabriel Synnaeve, and Baptiste Roziere. Getting the most out of your tokenizer for pre-training and domain adaptation. *arXiv preprint arXiv:2402.01035*, 2024.

[11] Nisansa De Silva. Survey on publicly available sinhala natural language processing tools and research. *arXiv preprint arXiv:1906.02358*, 2019.

[12] Polyglots FYP. Madlad_culturax_cleaned dataset. `https://huggingface.co/datasets/polyglots/MADLAD_CulturaX_cleaned`, 2025. Hugging Face model card.

[13] Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

[14] Omer Goldman, Avi Caciularu, Matan Eyal, Kris Cao, Idan Szpektor, and Reut Tsarfaty. Unpacking tokenization: Evaluating text compression and its correlation with model performance. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2274–2286, 2024.

[15] Hla Hla Htay and Kavi Narayana Murthy. Myanmar word segmentation using syllable level longest matching. In *Proceedings of the 6th Workshop on Asian Language Resources*, 2008.

[16] Kartik, Sanjana Soni, Anoop Kunchukuttan, Tanmoy Chakraborty, and Md Shad Akhtar. Synthetic data generation and joint learning for robust code-mixed translation, 2024.

[17] Sneha Kudugunta, Isaac Caswell, Biao Zhang, Xavier Garcia, Christopher A. Choquette-Choo, Katherine Lee, Derrick Xin, Aditya Kusupati, Romi Stella, Ankur Bapna, and Orhan Firat. Madlad-400: A multilingual and document-level large audited dataset, 2023.

[18] Zin Maung Maung and Yoshiki Mikami. A rule-based syllable segmentation of myanmar text. In *Proceedings of the IJCNLP-08 Workshop on NLP for Less Privileged Languages*, 2008.

[19] Meta. Llama 4 scout 17b 16e instruct. `https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct`, 2025. Hugging Face model card.

[20] OpenAI. tiktoken: fast BPE tokenizer for OpenAI models (o200k_base). `https://github.com/openai/tiktoken`, 2024. Accessed: 2026.

[21] Aleksandar Petrov, Emanuele La Malfa, Philip Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. *Advances in neural information processing systems*, 36:36963–36990, 2023.

[22] Shashini Y. Senanayake, K.T.P.M Kariyawasam, and Prasanna S. Haddela. Enhanced tokenizer for sinhala language. In *2019 National Information Technology Conference (NITC)*, pages 84–89, 2019. doi: 10.1109/NITC48475.2019.9114420.

[23] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 1715–1725, 2016.

[24] Muxin Tian. Cc100-sinhala filtered subset. `https://huggingface.co/datasets/realtmxi/CC100-sinhala`, 2025. Hugging Face model card.

[25] Menan Velayuthan and Kengatharaiyer Sarveswaran. Egalitarian language representation in language models: It all begins with tokenizers. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5987–5996, 2025.

[26] Ruvan Weerasinghe, Asanka Wasala, and Kumudu Gamage. A rule based syllabification algorithm for sinhala. In *International Conference on Natural Language Processing*, pages 438–449. Springer, 2005.